FIAP

FIAP GRADUAÇÃO

DISRUPTIVE ARCHITECTURES: IOT, IOB E GENERATIVE AI

Prof. Me. Renan Rodrigues Mendes

profrenan.mendes@fiap.com.br

Ferramentas

https://www.arduino.cc/reference/pt/

https://docs.arduino.cc/

https://www.tinkercad.com/

https://wokwi.com/

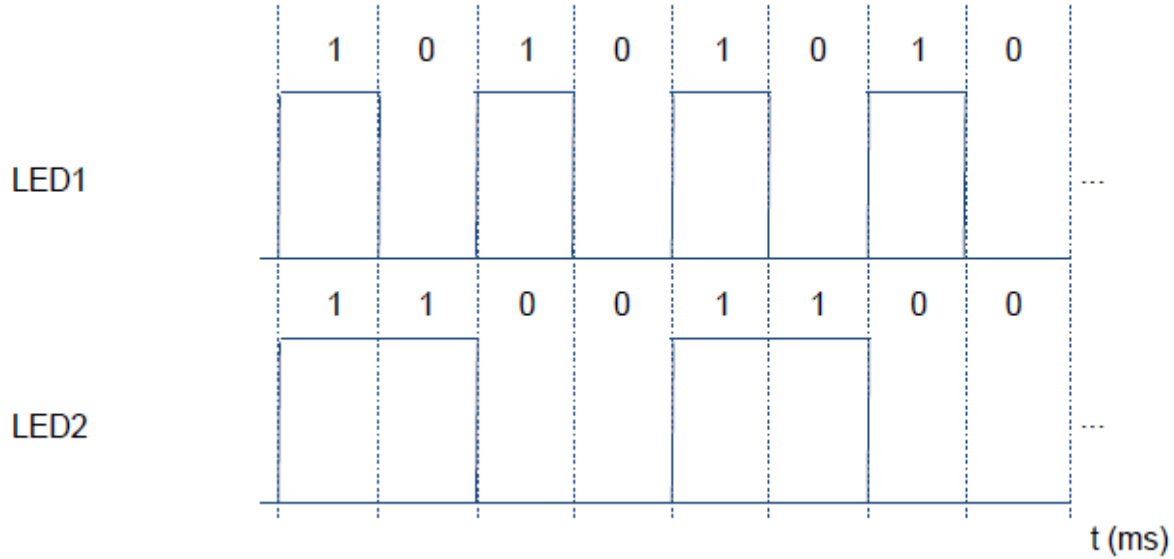
Exercícios

- Ampliar o circuito, utilizando mais portas e controlando mais LEDs;

- Sincronizar os LEDs, para que sejam habilitados e desabilitados sequencialmente (exercício semáforo)

- Agora, faça os LEDs acenderem conforme a carta de tempo abaixo. Onde:

1 = nível lógico alto (HIGH)
0 = nível lógico baixo (LOW)
delay = 500ms



Variáveis

- Variáveis são lugares (posições) na memória principal que servem para armazenar dados.
- As variáveis são acessadas através de um identificador único.
- O conteúdo de uma variável pode variar ao longo do tempo durante a execução de um programa.
- Uma variável só pode armazenar um valor a cada instante.

Tipos de Dados

- Tipos de dados definem:

  - A quantidade de memória que uma variável ou constante irá ocupar;
  - As operações que podem ser executadas sobre uma variável ou constante de determinado tipo;
  - A faixa de valores que uma variável ou constante pode armazenar;

Tipos de Dados

- Tipos de Variáveis no Arduino

Tipo	Definição
void	Indica tipo indefinido. Usado geralmente para informar que uma função não retorna nenhum valor.
boolean	Os valores possíveis são true (1) e false (0). Ocupa um byte de memória.
char	Ocupa um byte de memória. Pode ser uma letra ou um número. A faixa de valores válidos é de -128 a 127.
unsigned char	O mesmo que o **char**, porém a faixa de valores válidos é de 0 a 255.
byte	Ocupa 8 bits de memória. A faixa de valores é de 0 a 255.
int	Armazena números inteiros e ocupa 16 bits de memória (2bytes). A faixa de valores é de -32.768 a 32.767.
unsigned int	O mesmo que o **int**, porém a faixa de valores válidos é de 0 a 65.535.
word	O mesmo que um **unsigned int**.

Operadores

- Em uma linguagem de programação existem vários operadores que permitem operações do tipo:
  - Aritmética
  - Relacional
  - Lógica
  - Composta

Operadores

- Operadores aritméticos

Símbolo	Função
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
%	Módulo (resto da divisão inteira)

Operadores

- Operadores relacionais

Símbolo	Função
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual
==	Igual
!=	Diferente

Operadores

- Operadores lógicos

Símbolo	Função
&&	E (and)
| |	OU (or)
!	Não (not)

Operador de Atribuição

A atribuição de valores a variáveis e constantes é feita com o uso do **operador de atribuição (=).**

O operador de atribuição coloca o valor situado à sua direita dentro do objeto localizado à sua esquerda.

- Exemplos:

```
int valor = 100;
const float pi = 3.14;
```

Operadores

- Usando o operador de atribuição

atribuicao

```
int numero = 1; // inicialização

void setup()
{
  Serial.begin(9600);
  Serial.print("A variavel 'numero' vale: ");
  Serial.println(numero);
  delay(2000);
  numero = 5; // atribuição
  Serial.print("Agora a variavel 'numero' vale: ");
  Serial.println(numero);
  delay(2000);
}

void loop()
{
  numero = numero + 1; // atribuição
  Serial.print("Agora a variavel 'numero' vale: ");
  Serial.println(numero);
  delay(2000);
}
```

Monitor Serial

- O monitor serial é utilizado para comunicação entre o Arduino e o computador (PC).

- O monitor serial pode ser aberto no menu tools opção serial monitor, ou pressionando as teclas CTRL+SHIFT+M.

- As principais funções do monitor serial são: begin(), read(), write(), print(), println() e available().

Monitor Serial

- Algumas funções bastante usadas:

- begin(): inicializa a comunicação entre o Arduino e um computador;
- read(): recebe caracteres inseridos no monitor serial;
- print(): imprime caracteres no monitor serial;
- println(): imprime caracteres no monitor serial, mas causa uma quebra de linha no final;
- available(): retorna o número de bytes disponíveis no buffer de leitura do monitor serial.

Monitor Serial

- Imprimindo uma mensagem no monitor serial

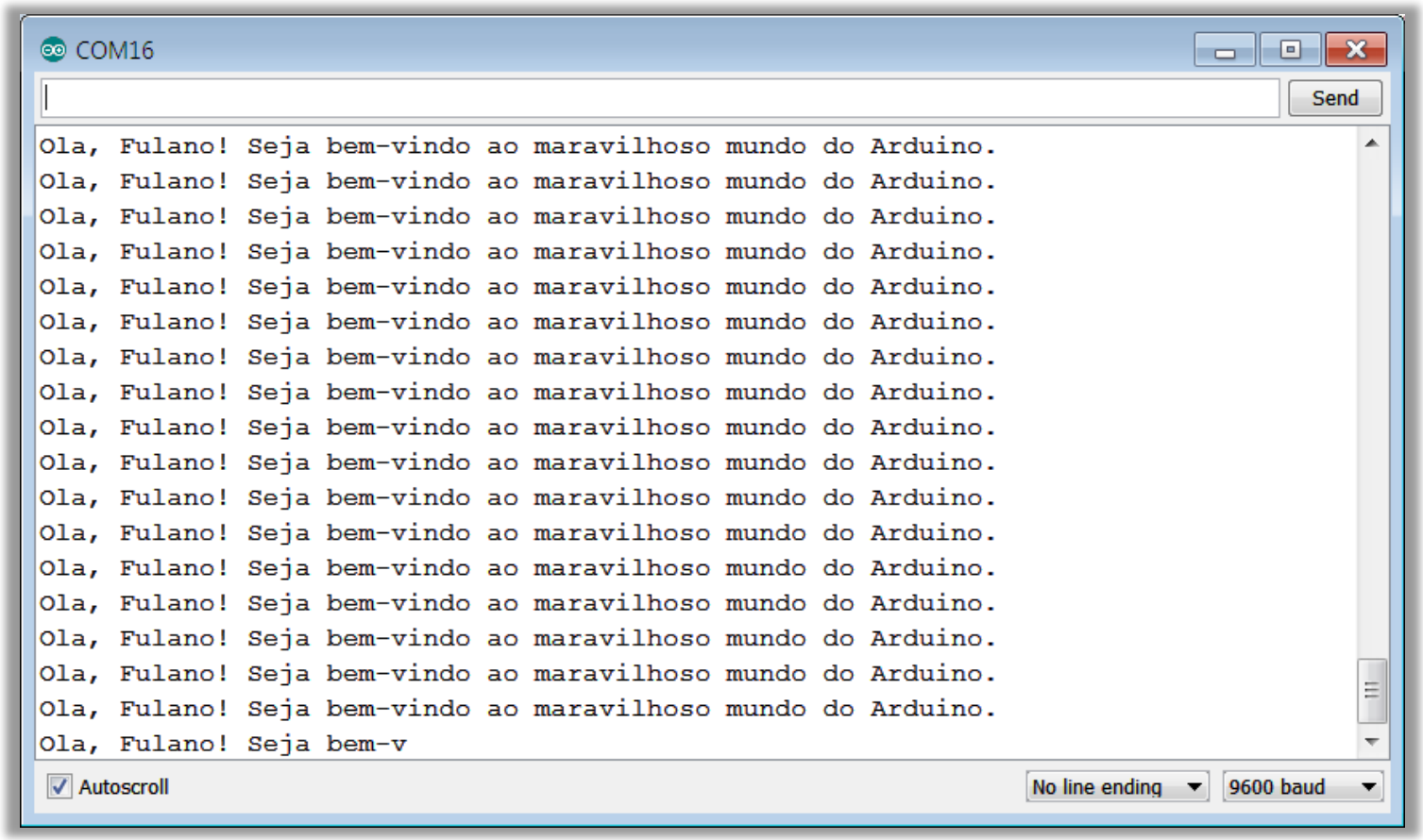
monitor_serial §

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  Serial.print("Ola, Fulano! Seja bem-vindo ao ");
  Serial.println("maravilhoso mundo do Arduino.");
}
```

Monitor Serial

- Saída no monitor serial



Comandos de Repetição

- Muitas vezes é necessário repetir uma determinada instrução ou conjunto de instruções.

- Os comandos de repetição mantêm em um "laço" uma instrução ou conjunto de instruções enquanto uma condição estiver sendo satisfeita.

- O Arduino possui 3 comandos de repetição:
  - while ( )
  - for (; ;)
  - do while ( )

Comandos de Repetição

- while ( )
  - Este comando de repetição avalia uma expressão no início do laço e, caso esta seja verdadeira, a(s) instrução(ções) dentro do "laço" permanecem sendo executadas.
  - Sintaxe do comando while:

```
while (expr) {
  cmd;
}
```

  - onde:
    - *expr* – é uma expressão que pode ser lógica, relacional ou aritmética.
    - *cmd* – um ou mais comandos a serem repetidos.
  - A permanência de execução do "laço" é garantida enquanto a expressão for verdadeira.

Como a avaliação da expressão é realizada no início do laço, pode ser que o *cmd* não execute nenhuma vez.

Comandos de Repetição

- while ( )

while

```
char ch;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  while (Serial.available() == 0) {
    Serial.println("Informe um caractere.");
    delay(300);
  }

  ch = Serial.read();
  Serial.print("ch = ");
  Serial.println(ch);
  delay(2000);
}
```

Comandos de Repetição

- while ( )

semaforo

```
/*********************************
* Nome do Programa: Semáforo     *
*                                *
* Descrição:                     *
*    Mantém o sinal aberto até   *
*    que um pedestre solicite a  *
*    travessia. Então fecha o    *
*    sinal por alguns segundos.  *
*********************************/

const int BOTAO = 6;
const int VERDE = 8;
const int AMARELO = 9;
const int VERMELHO = 10;

void setup()
{
  pinMode(VERDE, OUTPUT);
  pinMode(AMARELO, OUTPUT);
  pinMode(VERMELHO, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
}

void loop()
{
  // abre o sinal
  digitalWrite(VERDE, HIGH);

  // aguarda pedestre
  while (digitalRead(BOTAO))
    ; // instrução vazia

  // muda para o amarelo
  digitalWrite(VERDE, LOW);
  digitalWrite(AMARELO, HIGH);
  delay(4000);

  // fecha o sinal
  digitalWrite(AMARELO, LOW);
  digitalWrite(VERMELHO, HIGH);
  delay(8000);

  // abre novamente
  digitalWrite(VERMELHO, LOW);
}
```

Comandos de Repetição

- while ( )

while_2

```
int contador;

void setup()
{
  Serial.begin(9600);

  contador = 0;
  while (contador < 10) {
    Serial.println("Oficina de Robotica");
    contador++;
  }
}

void loop() {}
```

Comandos de Repetição

- for (; ;)
  - Este comando de repetição deve ser utilizado quando se sabe a quantidade de vezes que uma determinada instrução deve ser executada.

  - O comando for permite que escrevamos de forma mais organizada os laços de repetição baseados em um contador.

  - Sintaxe do comando for:

```
for (inicialização; condição; incremento) {
  cmd;
}
```

  - onde:

    inicialização – inicialização do contador;

    condição - é uma expressão relacional ou lógica;

    incremento - atualização do contador.

Comandos de Repetição

- for (; ;)

for

```
int vezes = 10; // qtde de vezes que a mensagem será impressa
int contador; // irá contar quantas vezes a mensagem já foi impressa

void setup()
{
  Serial.begin(9600);

  for (contador = 0; contador < vezes; contador++) {
    Serial.println("Testando o comando de repeticao for()");
  }
}

void loop()
{

}
```

Comandos de Repetição

- **Nota**
  - É possível declarar o contador dentro do cabeçalho do laço for.

```
for (int i = 0; i < 50; i++) {
  Serial.println("Oficina de Robotica");
}
```

Comandos de Repetição

- do while ( )
  - Semelhante ao comando while. A única diferença é que a condição é testada no final do laço.
  - Sintaxe do comando do while:

```
do {
  cmd;
} while (expr) ;
```

Como a avaliação da expressão é realizada no final do laço, é garantido que o *cmd* será executado pelo menos uma vez.

Comandos de Seleção

- Em vários momentos em um programa precisamos verificar uma determinada condição afim de selecionar uma ação ou ações que serão executadas.
- Um comando de seleção também é conhecido por desvio condicional, ou seja, dada um condição, uma parte do programa é executada.

- Os <u>comandos de seleção podem ser</u> do tipo:
  - Seleção simples
  - Seleção composta
  - Seleção de múltipla escolha

Comandos de Seleção

- Seleção simples
  - Um comando de seleção simples avalia uma condição, ou expressão, para executar uma ação ou conjunto de ações.
  - No Arduino o comando de seleção simples é:

```
if (expr)   {
    cmd
}
```

  - onde:

expr – representa uma expressão a ser avaliada que po ser do tipo lógica, relacional ou aritmética. O resultado avaliação de de uma expressão é sempre um valor lógico. ***cmd*** – comando(s) a da ser executado.

Comandos de Seleção

- Seleção simples
  - Acendendo leds pelo monitor serial

selecao_simples

```
#define LED_VERDE    5
#define LED_AMARELO  6
#define LED_VERMELHO 7

char opcao;

void setup()
{
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARELO, OUTPUT);
  pinMode(LED_VERMELHO, OUTPUT);
  Serial.begin(9600);
}

void loop()
{
  opcao = Serial.read();

  if (opcao == 'G')
    digitalWrite(LED_VERDE, HIGH);
  if (opcao == 'Y')
    digitalWrite(LED_AMARELO, HIGH);
  if (opcao == 'R')
    digitalWrite(LED_VERMELHO, HIGH);
}
```

Comandos de Seleção

- Seleção simples
  - Verificando se há caracteres no buffer de leitura

serial_available

```
char caractere;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  if (Serial.available()) {
    caractere = Serial.read();
    Serial.print("Caractere lido: ");
    Serial.println(caractere);
  }
}
```

Comandos de Seleção

- Seleção composta
  - Um comando de seleção composta é complementar ao comando de seleção simples.
  - O objetivo é executar um comando mesmo que a expressão avaliada pelo comando *if (expr)* retorne um valor falso.
  - No Arduino o comando de seleção composta é:

```
if (expr) {
    cmd;
}
else {
    cmd;
}
```

  - onde:
    - ***expr*** – representa uma expressão a ser avaliada que pode ser do tipo lógica, relacional ou aritmética. O resultado da avaliação de uma expressão é sempre um valor lógico.
    - ***cmd*** – comando(s) a ser executado.

Comandos de Seleção

- Seleção composta
  - Verificando se há caracteres no buffer de leitura

selecao_composta_01

```
char caractere;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  if (Serial.available()) {
    caractere = Serial.read();
    Serial.print("Caractere lido: ");
    Serial.println(caractere);
  }
  else {
    Serial.println("Insira um caractere!");
  }
  delay(400);
}
```

Comandos de Seleção

- Seleção composta (Comandos if aninhados)
  - Acendendo e apagando leds pelo monitor serial

selecao_composta_02

```
#define LED_VERDE     2
#define LED_AMARELO   3
#define LED_VERMELHO  4

char opcao;

void setup()
{
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARELO, OUTPUT);
  pinMode(LED_VERMELHO, OUTPUT);
  Serial.begin(9600);
}
```

```
void loop()
{
  if (Serial.available()) {
    opcao = Serial.read();

    if (opcao == 'G')
      digitalWrite(LED_VERDE, HIGH);
    else if (opcao == 'g')
      digitalWrite(LED_VERDE, LOW);
    else if (opcao == 'Y')
      digitalWrite(LED_AMARELO, HIGH);
    else if (opcao == 'y')
      digitalWrite(LED_AMARELO, LOW);
    else if (opcao == 'R')
      digitalWrite(LED_VERMELHO, HIGH);
    else if (opcao == 'r')
      digitalWrite(LED_VERMELHO, LOW);
    else
      Serial.println("Opcao Invalida.");
  }
}
```

Comandos de Seleção

- Seleção de múltipla escolha
  - Na seleção de múltipla escolha é possível comparar vários valores.
  - No Arduino o comando de seleção de múltipla escolha é:

```
switch (valor) {
    case x: cmd1;
            break;
    case y: cmd2;
            break;
    default: cmd3;
}
```

  - onde:
    - *valor* – é um dado a ser avaliado. É representado por uma variável de memória.
    - cmd_x – comando a ser executado.

      case – indica a opção a ser executada.

      default – comando padrão que deverá ser executado se nenhuma outr escolha (*case*) tiver sido selecionada.

Comandos de Seleção

- Seleção de múltipla escolha
  - Acendendo e apagando leds pelo monitor serial

selecao_multipla_escolha

```
#define LED_VERDE     2
#define LED_AMARELO   3
#define LED_VERMELHO  4

char opcao;

void setup()
{
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARELO, OUTPUT);
  pinMode(LED_VERMELHO, OUTPUT);
  Serial.begin(9600);
}
```

```
void loop()
{
  if (Serial.available()) {
    opcao = Serial.read();

    switch (opcao) {
      case 'G': digitalWrite(LED_VERDE, HIGH);
                break;
      case 'g': digitalWrite(LED_VERDE, LOW);
                break;
      case 'Y': digitalWrite(LED_AMARELO, HIGH);
                break;
      case 'y': digitalWrite(LED_AMARELO, LOW);
                break;
      case 'R': digitalWrite(LED_VERMELHO, HIGH);
                break;
      case 'r': digitalWrite(LED_VERMELHO, LOW);
                break;
      default:  Serial.println("Opcao Invalida.");
    }
  }
}
```

Exercícios

1) Familiarize-se com o Arduino IDE e o Monitor Serial, escrevendo um programa simples que imprime "Hello, World!" no Monitor Serial.

2) Crie um programa que recebe dois números inteiros do Monitor Serial, realiza operações matemáticas básicas (adição, subtração, multiplicação e divisão) e exibe os resultados no Monitor Serial.

3) Escreva um programa que receba um número inteiro do Monitor Serial e, usando estruturas de controle, verifique se o número é par ou ímpar, positivo ou negativo e imprima o resultado no Monitor Serial.

4) Utilizando um laço for(), faça dois LEDs piscarem simultaneamente 6 vezes, na inicialização.

5) Crie um programa que receba um comando representado pelas letras 'F' ou ‘T’ via serial. O sistema deve interpretar esse comando e acionar LEDs de acordo com as seguintes regras: Se o comando recebido for 'F', os LEDs 1 e 3 devem acender. Se o comando recebido for 'T', os LEDs 2 e 4 devem acender.